UNIT 8 60 min

Tuples and Sets

Fixed-size tuples for grouped values, and sets for unique items with fast membership tests.

By the end of this unit you will be able to

- Create tuples and explain why they are immutable
- Unpack a tuple into separate variables and return several values from a function
- Create sets, explain why they hold no duplicates, and use union, intersection, and difference
- Choose between a list, a tuple, and a set for a task

Tuples: fixed groups

A **tuple** is an ordered collection written with parentheses. Like a list, it is indexed and sliced. Unlike a list, it is **immutable**: once created, you cannot add, remove, or replace items. Use a tuple when the group should not change, such as an (x, y) coordinate or an RGB color.

tuples.py

```
point = (4, 7)
color = (255, 128, 0)
print(point[0])
print(len(color))
```

Watch out

A single-item tuple needs a trailing comma: `(5,)` is a tuple, but `(5)` is just the number 5.

Unpacking

You can assign each item of a tuple to its own name in one line. This is called **unpacking**. The number of names must match the number of items.

unpacking.py

```
x, y = (3, 4)
print(x + y)

def min_max(nums):
    return min(nums), max(nums)

low, high = min_max([7, 2, 9, 4])
print(low, high)
```

Python functions return multiple values by returning a tuple. The caller unpacks it. This is the most common reason you will see tuples in real code.

Sets: unique items

A **set** is an unordered collection of unique values, written with curly braces. Adding a duplicate does nothing. Checking whether an item is in a set is very fast, even for large sets.

sets.py

```
visited = {1, 2, 2, 3, 3, 3}
print(visited)
print(len(visited))
visited.add(4)
print(4 in visited)
```

> **Watch out**
> `{}` creates an empty **dictionary**, not a set. Use `set()` for an empty set. Also, the order of items in a set is not guaranteed, so do not rely on it.

Set operations

Operation	Symbol	Method	Meaning
Union	`a | b`	`a.union(b)`	Items in either set

Operation	Symbol	Method	Meaning
Intersection	`a & b`	`a.intersection(b)`	Items in both sets
Difference	`a - b`	`a.difference(b)`	Items in a but not in b

set_ops.py

```
math_class = {"Ana", "Ben", "Cy"}
art_class = {"Ben", "Dee"}

print(math_class | art_class)
print(math_class & art_class)
print(math_class - art_class)
```

Choosing a collection

Need	Use	Why
An ordered sequence that changes	list	Indexable, mutable, allows duplicates
A fixed group, such as coordinates	tuple	Immutable; safe to pass around
Unique items, fast membership tests	set	Duplicates removed automatically
Labels that map to values	dict (Unit 9)	Look up by key instead of position

Tip

To remove duplicates from a list while keeping it a list: `unique = list(set(items))`. The order may change, so use this only when order does not matter.

Predict the output THINK FIRST

Watch for the single-item tuple and for the empty set.

```
print((5))
```

Answer: `5`
Parentheses alone do not make a tuple.

```
print((5,))
```

Answer: `(5,)`

```
t = (1, 2)
t[0] = 9
```

Answer: `TypeError`
Tuples cannot be changed.

```
x, y = (3, 4)
print(x + y)
```

Answer: `7`

```
print({1, 2, 2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2} | {2, 3})
```

Answer: `{1, 2, 3}`

```
print({1, 2, 3} & {2, 3, 4})
```

Answer: `{2, 3}`

```
print(len(set("banana")))
```

Answer: `3`
The unique letters are b, a, n.

```
print({} == set())
```

Answer: `False`
`{}` is an empty dictionary.

Practice

▼ **Exercise 1 — Shared clubs**

Two clubs have these members: `chess = ["Ana", "Ben", "Cy", "Ben"]` and `robotics = ["Ben", "Dee", "Ana"]`. Print the names that appear in **both** clubs, without duplicates.

▼ **Hint**

Convert each list to a set, then use `&`.

▼ **Show solution**

Solution

```
chess = ["Ana", "Ben", "Cy", "Ben"]
robotics = ["Ben", "Dee", "Ana"]
shared = set(chess) & set(robotics)
print(shared)
```

▼ Exercise 2 — Unpack a return

Write a function `stats(nums)` that returns a tuple of `(smallest, largest, count)`. Call it with `[4, 8, 1, 6]` and unpack the result into three variables. Print each.

▼ Show solution

Solution

```
def stats(nums):
    return min(nums), max(nums), len(nums)

smallest, largest, count = stats([4, 8, 1, 6])
print(smallest)
print(largest)
print(count)
```

Discussion

1. Why might a programmer choose a tuple over a list even when both would work? What is the benefit of "can't change it"?
2. A set throws away duplicates and order. When is that a feature, and when is it a bug?
3. Pick one real data source (a roster, a list of coordinates, a log of website visits) and say which collection type you would use and why.

▼ Facilitator notes

Set iteration order is an unfixed detail of the interpreter. If a student asks why `print({"b", "a"})` shows a different order from one run to the next, that is the answer. Do not promise an order.

Key terms

Tuple

An immutable, ordered collection written in parentheses, such as `(4, 7)`.

Immutable

Cannot be changed after creation. Tuples, strings, and numbers are immutable.

Unpacking

Assigning each item of a sequence to a separate name in one statement, such as `x, y = point`.

Set

An unordered collection of unique values, written in curly braces or created with `set()`.

Union

The set of items in either of two sets. Written `a | b`.

Intersection

The set of items in both sets. Written `a & b`.

Difference

The items in the first set that are not in the second. Written `a - b`.

Membership test

Checking whether a value is in a collection with the `in` operator.

See all terms in the glossary →